Chapter 4

Aesthetics

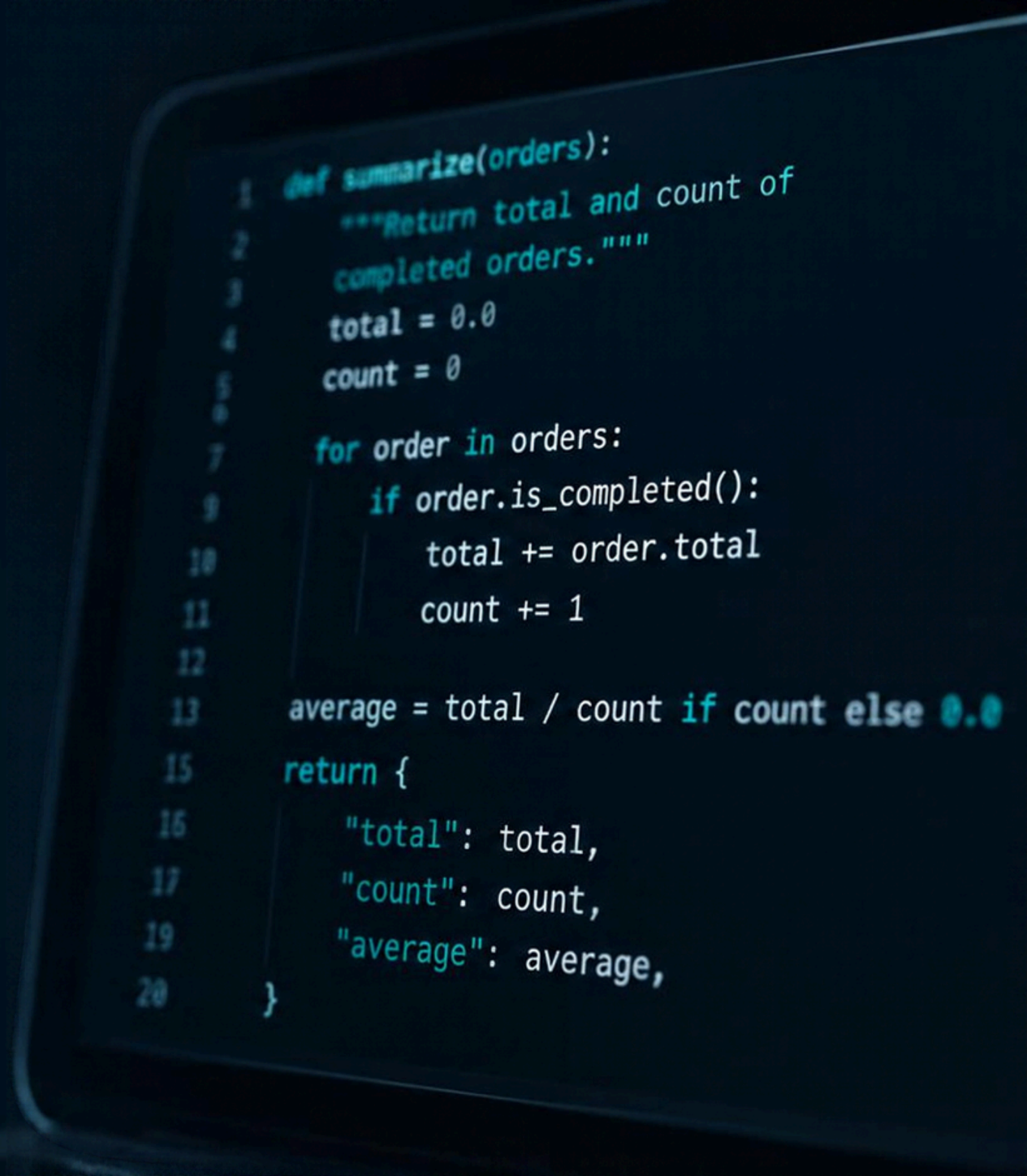
Member :

Kuong Sivhoung

Khay Lina

Yory Sreykhuoch

How spacing, alignment, and ordering make code easier to scan and understand.



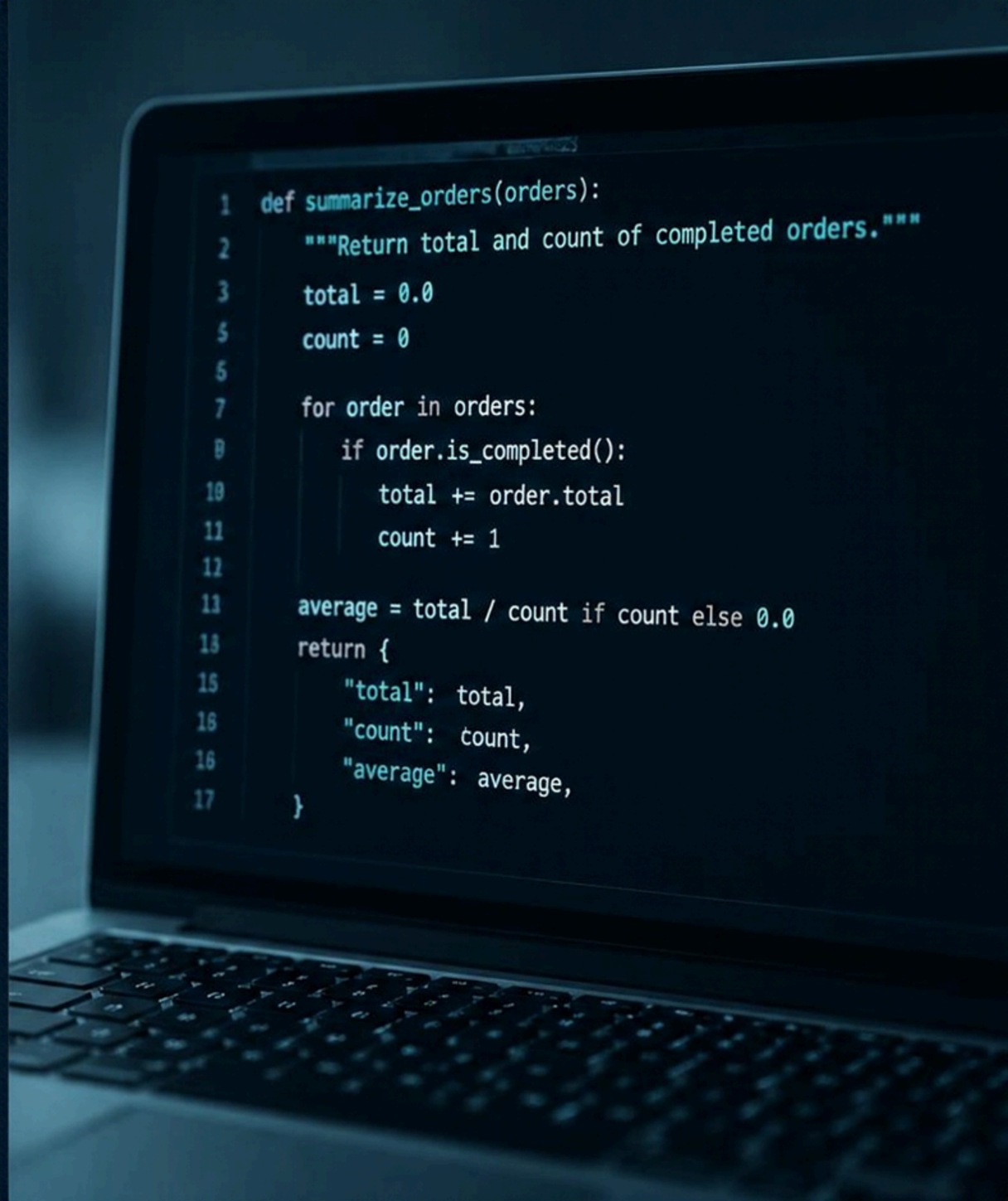
Why Aesthetics Matter

Programmers spend most of their time reading code.

Readable code is easier to skim.

Good formatting helps readers locate important details.

Pretty code is useful when it serves understanding.



Three Core Principles

01	Use a consistent layout.
02	Make similar code look similar.
03	Group related lines into blocks.

Keep Line Breaks Consistent

Similar code should look similar. Aligned data reduces mental load.

AVOID THIS

```
def area(width,height,
unit="m"):   return {
"width":   width,   "height":
height,   "unit":unit,
"area":   width * height,   }
```

→

PREFER THIS

```
# width: horizontal length (meters)
# height: vertical length (meters)
# unit: measurement unit label
# returns area in square units

def area(width,height,
unit="m"):   return {
"width":   height,
"height":   unit,     "unit":
width * height,   }
```

Keep related values easy to scan

Preserve the same shape for similar code

Use a short comment to explain order and units

Consistency builds trust and speeds understanding

Use Helpers to Remove Irregularity

```
def add cases():
  calc = Calculator()
  return [
    (2, 3, 5),
    (0, 10, 10),
    (-4, -6, -10),
  ]
```

Use Helpers to Remove Irregularity

BEFORE · Repeated logic

```
def test_add():
  calc = Calculator()
  assert calc.add(2, 3) == 5

def test_add_zero():
  calc = Calculator()
  assert calc.add(0, 10) == 10

def test_addNegative():
  calc = Calculator()
  assert calc.add(-4, -6) == -10
```

AFTER · Shared test logic

```
def add cases():
  return [(2, 3, 5), (0, 10, 10),

   ( -4,  -6,  -10)]

for a, b, expected in add cases():
  calc = Calculator()
  assert calc.add(a, b) == expected
```

One **helper** organizes test cases; one **loop** applies the same check.

Align Columns When It Helps

Aligned columns create **visual handrails**.

Differences and errors become **easier to spot.**

Useful for **related variables**, **records**, and **tables**.

Align for clarity, **not at the cost of noisy diffs**.

Choose a Meaningful, Consistent Order

Follow an order readers can predict.

Match the real-world or data order.

Use importance or alphabetical order when relevant.

Keep the same order across related code.

Group Declarations into Blocks

Organize related code together by responsibility.

Use clear headings or spacing to separate concerns.

Readers see the structure first, then the details.

It's easier to scan, navigate, and maintain.

```
# --- Request handlers ---
def list items(request):
    ...
def createItem(request):
    ...
def updateItem(request, item_id):
    ...
def deleteItem(request, item_id):
    ...

# --- Utility functions ---
def parseParams(request):
    ...
def validate payload(data):
    ...

# --- Database helpers ---
def getitem(item_id):
    ...
def saveItem(data):
    ...
def removeItem(item_id):
    ...
```

Break Code into Paragraphs

Group related steps into clear logical stages.

Use blank lines to create visual boundaries.

Add brief comments so readers can skim with confidence.

Readable structure makes intent obvious at a glance.

```
1   def total_orderValue (orders):
2       # Stage 1: gather valid orders
3       valid = []
4       for order in orders:
5           if not order.cancelled:
6
7       # Stage 2: calculate each order's value
8       values = []
9       for order in valid:
10
11      # Stage 3: combine the totals
12      total = 0.0
13      for value in values:
14
15      # Stage 4: return the result
16      return total
17
```

Consistency Is the Main Rule

Consistent
Keep your style steady throughout the code.

Aligned where useful
Line things up to show relationships.

Logically ordered
Arrange code to follow the flow of ideas.

Grouped with intent
Put related pieces together and separate concerns.

Easy to scan
Write so others can quickly find what matters.